

PyQCU volume24_sweep_multi_gpu_2 — 24×24×24×72 大格子 Clover MultiGrid 真实加速比分析

opencode analy

2026 年 9 月 27 日

摘要

本报告针对 PyQCU `pyqcu/testing/qcu/multigrid/volume24/sweep_multi_gpu` 在统一格子 $24 \times 24 \times 24 \times 72$ 上对 CUDA C++ 多层 Clover MultiGrid 的真实加速比进行只读分析。输入数据为 `data/logs/nullvec_cache/L24x24x24x72_lv1_E24_nvi20_t0.01.h5`、`pyqcu/testing/qcu/multigrid/volume24/sweep_multi_gpu/bench_24x24x24x72.h5` (gate=1.0, 18 配置全收敛, best 1.107) 与 `pyqcu/testing/qcu/multigrid/volume24/sweep_multi_gpu/gauge_24x24x24x72.h5` 统一规范场。分析覆盖全库 842 文件, 三视角齐备: 主体结构(两层执行架构+多线程多卡)、各部分关系(参数协议 → 粗算子构建 → 求解 → 日志)、项目思路(小格子 > 2× 不可迁移至大格子的物理与工程权衡)。主题分析按 A 代码工作框架 15 步完整重现 V00 单卡与 P100×2 双卡的基准流程, 结论为 $24^3 \times 72$ 上 3L 加速比上限 ~ 1.17 , gate=1.0 下 ALL PASS, 与 `pyqcu/testing/logs/multigrid/legacy`、`pyqcu/testing/logs/multigrid/scaling` 图表类型完全对齐。

目录

1 仓库概览	1
1.1 目录结构	1
1.2 规模统计与 git 信息	1
2 主体结构	2
3 各部分关系	3
4 项目思路	4
4.1 设计意图	4
4.2 演进脉络	4
4.3 典型工作流	4
5 主题分析	4
5.1 工作全流程重现 (A 代码工作框架)	5
5.1.1 A1 目的与前期准备	5
5.1.2 A2 输入是什么	5

5.1.3	A3 输出应该是什么	5
5.1.4	A4 整体框架	5
5.1.5	A5 关键细节	5
5.1.6	A6 任务如何正确执行	6
5.1.7	A7 实际执行情况	6
5.1.8	A8 实际输出情况	6
5.1.9	A9 结果分析	7
5.1.10	A10 综合分析	10
5.1.11	A11 结尾总结	10
5.1.12	A12 结尾评价	10
5.1.13	A13 补充说明	10
5.1.14	A14 参考源	10
5.1.15	A15 附录与附件	10
6	关键代码/文档片段	11
7	本次大任务图表详览	13
8	本次大任务日志关键内容汇编	14
9	参考源清单	16
10	结论与局限	21

1 仓库概览

1.1 目录结构

```
1 PyQCU/  
2   pyqcu/ (纯 Python: dslash/solver/tools/cuda)  
3   cpp/cuda/qcu/ (C++ CUDA 后端 26 头 + src)  
4   pyqcu/testing/qcu/multigrid/volume24/sweep_multi_gpu/ (本任务工作目录, main.py 535 行)  
5   logs/test15_2/ (本任务产物, 23 PNG+7 TEX+4 TXT+2 H5)  
6   data/logs/nullvec_cache/ (共享粗算子 5.5GB+0.4GB)  
7   docs/ (本报告)
```

1.2 规模统计与 git 信息

本任务基线 dev79: gitshowdev79--stat 显示 pyqcu/testing/qcu/multigrid/volume24/sweep_multi_gpu/main.py 535 行 + pyqcu/tools/_multigrid.py 280 行, 消息 “局部化 stencil 构建 BatchedLocalSchur $W = 10$ ”。

指标	实测值
文档数 (*.md)	38
Python 文件数 (*.py)	127
Shell 脚本数 (*.sh)	19
总行数 (py)	41832
提交数 (git log)	342
标签数 (stab/dev/bug/test)	71

表 1: 仓库实测统计 (数据来源: find/wc/git 实测)

2 主体结构

层次	目录	职责
入口层	pyqcu/testing/qcu/multigrid/volume24/sweep_multi_gpu/main.py:22-35	子命令 build/bench/check- /mul- ti/re- port + define. py 参数 MultiGpuMultigrid 一 线程 一 卡, param- s/argv/set_ptrs 隔离
核心层	pyqcu/cuda/_multi_gpu.py:210-400	C++ V-cycle, fused 粗 求 解, PROF_SECTIONS BatchedLocalSchur W = 10, _schur_matvec_batch, build_stencil_local
核心层	cpp/cuda/qcu/include/lattice_clover_multigrid.h	
工具层	pyqcu/tools/_multigrid.py:517-650	

层次	目录	职责
工具层	pyqcu/tools/_io.py:130-210	h5py 持久化 save_dict_h5/load_tensors
配置层	pyqcu/cuda/define.py:15-45	params[54] /argv[7] /set_ ptrs[100], _SET_INDEX_++
参考层	refer/git-rep/DDalphaAMG/docs	MG 范 例, 宽版 粗算子 理论
文档层	logs/dev73/dev73_5.tex	dev73/dev74 图表模 板

表 2: 主体结构分层 (数据来源: 全库索引实测)

3 各部分关系

要点

依赖主链: gauge (seed=42) → Clover (kappa=0.1230) → Schur 算子 S → null 向量 (CudaSchurOp, $E = 24$, nvi=20) → 33-tensor stencil (BatchedLocalSchur $W = 10$) → 粗算子 H5 → MG 求解 (applyCloverMultigridQcu) → CONVERGENCE_HISTORY/PROF_SECTIONS → bench H5 → PNG/TEX/TXT

关系	链	说明
生成	main.py:224-345 → data/logs/nullvec_cache/L24*.h5	build 子命令, 窗口 $W = 10$, 中心 $c \pm 1$, diff=0
引用	main.py:137-160 → _multi_gpu.py:269-350	粗算子 H5 被主线程加载后拷至各卡 set_ptrs[30+4fl]
包含	define.py ↔ cpp/include/define.h	params 索引必须同步, 否则越界读

关系	链	说明
配置配对	<code>main.py:48-63</code> ↔ <code>pyqcu/testing/qcu/AGENTS.md:15-30</code>	$LAT=24^3 \times 72$, $MASS$ 0.05, $ATOL$ $1e-6$, DOF_LIST [12,24]
版本演进	<code>dev73</code> → <code>dev74</code> → <code>dev79</code> → <code>volume24_sweep_multi_gpu</code>	粗层 $E = 48 \rightarrow 24$, 局部化 $W = 10$, gate 分级 2.0→1.0

表 3: 各部分依赖与演进关系（数据来源: grep/read 实测）

4 项目思路

4.1 设计意图

为什么统一 $24^3 \times 72$ 且 gate 分级? 小格子 $8^3 \times 16$ 粗层 4^4 时 Galerkin 低频捕捉有效, MG $2.43 \times \text{logs/dev73/dev73_5.md:18-25}$; 大格子粗层 $12 \times 12 \times 12 \times 18$ 达 31104 点 $\times 24^2$ 稠密块, 粗算子质量下降且 $L1$ Schur BiStabCG 本身高效 (148 迭代, 2.808s), V-cycle 开销占比 $> 10\%$ 导致加速比上限 ~ 1.17 。故 `main.py:401-404` 按格子分级 gate, 大格子 1.0 (不慢于 $L1$ 即通过)。

4.2 演进脉络

局部化 $W = 10$ 的权衡: 全格 torch 批量 $\sim 22h$ / C++ 逐场 $\sim 178min$ 均不可行 `logs/test15_2/test15_2_report.md:15-25`; 窗口 $W = 10$ 使 c 块 $[2c, 2c + 2)$ 居中, 仅探中心 $c \pm 1$ 块, 与全格 $diff=0$ 且每点 54ms, 31104 点 $\sim 28min$, 连续切片比高级索引快 $86\times$ 。

4.3 典型工作流

`source./env.sh` → `bash./build.sh` → `pythonpyqcu/testing/qcu/multigrid/volume24/sweep_multi_gpu/main.pybuild` (31min) → `bench--pairs3--restarts152030--cts1001e31e5` (3min) → `check--filebench_24x24x24x72.h5` (gate 1.0) → `generate_assets.py` → `docs/analy_volume24_sweep_multi_gpu*.tex`。

项目思路

核心总结: 小格子 $> 2\times$ 源于粗层小且 $L1$ 弱, 大格子 $12^3 \times 18$ 时粗层大且 $L1$ 强, MG 收益被 V-cycle 开销抵消, 故 `volume24_sweep_multi_gpu` 以正确性与不劣化为通过标准, 产物对 `dev73/dev74` 形成闭环。

5 主题分析

5.1 工作全流程重现 (A 代码工作框架)

5.1.1 A1 目的与前期准备

本工作目标为 稳定复现 dev79 在 $24^3 \times 72$ 上的单卡 V100 与双卡 P100 $\times 2$ 基准, 并补充与 dev73/dev74 同类型全部图表日志。前置条件: pyqcu/cuda/define.py:19-35 参数协议同步, cpp/cuda/qcu/libqcu.so 已构建, data/logs/nullvec_cache 存在 4 缓存, 3 GPU (torch 0=V100,1/2=P100) 可见 pyqcu/testing/qcu/AGENTS.md:22-35。

5.1.2 A2 输入是什么

输入	路径	含义
粗算子	data/logs/nullvec_cache/L24x24x24x72_lv1_E24_nvi20_t0.01.h5	lonv,hnn,hdg,sit 5.5GB
粗算子	data/logs/nullvec_cache/L24x24x24x72_lv2_E24_nvi20_t0.01.h5	lv2 0.4GB $6^3 \times 9$
基准	pyqcu/testing/qcu/multigrid/volume24/sweep_multi_gpu/bench_24x24x24x72.h5	gate1.0 18MG+L1+ref 19 配置
规范场	pyqcu/testing/qcu/multigrid/volume24/sweep_multi_gpu/gauge_24x24x24x72.h5	U_full clover kappa0.123
参数	main.py:49-63	LAT $24^3 \times 72$ DOF[12,24]

表 4: 输入清单 (数据来源: h5py 实测 + main.py:49-63)

5.1.3 A3 输出应该是什么

预期产物: logs/test15_2/test15_2_conv_*.png 等 13 PNG, logs/test15_2/test15_2_tbl_*.tex 5 TEX, logs/test15_2/mg_bench_out.txt 等 4 TXT, logs/test15_2/test15_2_bench.json 总结, docs/analy_volume24_sweep_multi_gpu_*.pdf 本报告, 最终 volume24_sweep_multi_gpu 标签。

5.1.4 A4 整体框架

main.py:22-35 5 子命令: build(局部化) $\rightarrow$ bench(V100 3 $\times$ 中位) $\rightarrow$ check(gate) $\rightarrow$ multi(P100 $\times 2$) $\rightarrow$ report; generate_assets.py 复用 dev73 亮色调色板与 define.py 常量, 输出落 pyqcu/testing/logs/volume24/sweep_multi_gpu 以对齐 dev73/dev74。

5.1.5 A5 关键细节

关键函数: pyqcu/tools/_multigrid.py:517 __schur_matvec_batch 分块 0.5GB, pyqcu/cuda/_multi_gpu.py:483-525 fast-path 避 OOM。易错点: main.py:336 tuple 误传已修复。

```

3 # ===== Schur 33-tensor stencil build (Galerkin 粗网格算子) =====
4 # 由 pyqcu/testing/qcu/multigrid/legacy/mg_stencil_build.py 与 test12/main.py::build_stencil_mt 合并迁移。
5
6 PAIRS = [(0, 1), (0, 2), (0, 3), (1, 2), (1, 3), (2, 3)] # (d1,d2) with d1<d2
7 SIGN = [1, -1]
8
9
10 def _bistabcg_batch(b_batch, matvec_batch, tol=1e-2, max_iter=2000,
11                    if_rtol=False):
12     """批量 BiCGStab: 同时对 batch 维 (B) 内的全部右端求解。
13
14     b_batch: [B, e, X, Y, Z, T/2]; matvec_batch: 批量 matvec (同形 → 同形)。
15     标量 (rho/alpha/omega/beta) 按批独立 ([B] 向量), 迭代直到全部批次收敛
16     或 max_iter。breakdown (rho/rtv/tts 0) 批次用 eps 保护避免除零,
17     最终未收敛批次由调用方回退随机 (与 give_null_vecs_mt 语义一致)。
18     if_rtol=True 时 tol 为相对容差 (以各批初始 ||b|| 为基准)。
19     2026-08-15: null 向量生成批量化 —— 16x16x16x32 lv2 的 48 个右端
20     (196608 未知数) 一次迭代 (C++ 逐场 40min+ → torch 批量分钟级)。
21     """
22     B = b_batch.shape[0]
23     x = torch.zeros_like(b_batch)
24     r = b_batch.clone()
25     r_norm = torch.linalg.norm(r.reshape(B, -1), dim=1)
26     b_norm = r_norm.clone()

```

5.1.6 A6 任务如何正确执行

```

1 source ./env.sh
2 python pyqcu/testing/qcu/multigrid/volume24/sweep_multi_gpu/main.py bench --pairs 3 --restarts 15 20 30 --cts 100 1000
   ↪ 100000 --cmi 3 # V00 单卡
3 python pyqcu/testing/qcu/multigrid/volume24/sweep_multi_gpu/main.py check --file bench_24x24x24x72.h5 # gate 1.0
4 python pyqcu/testing/qcu/multigrid/volume24/sweep_multi_gpu/main.py multi --levels 2 --restart 10 --ct 1e5 --cmi 15 #
   ↪ P100x2
5 python pyqcu/testing/qcu/multigrid/volume24/sweep_multi_gpu/generate_assets.py # 补充 dev73/dev74 同类型图表日志

```

5.1.7 A7 实际执行情况

实测 V100 bench: L1 median 2.808s (2.788/2.808/2.821), ref BiStabCG 3.197s, 18 配置各 3× 中位, 总计 ~58 求解 ×3s ≈174s + 粗算子加载。日志 logs/clover_multigrid.log:64022-67445 含 115 次 Lt_full=72 MG 启动, PROF_SECTIONS 均记录 fine_iter/vcycle/coarse_solve。P100×2 multi 因原 _multi_gpu.py:486 额外 CudaSchurOp 导致 27GB OOM, 已通过 cache_hit fast-path 修复 pyqcu/cuda/_multi_gpu.py:483-525。

5.1.8 A8 实际输出情况

产物	路径	实测值
bench H5	pyqcu/testing/qcu/multigrid/volume24/ sweep_multi_gpu/bench_24x24x24x72.h5	18MG+L1+ref=19 gate1.0 best1.107
multi H5	pyqcu/testing/qcu/multigrid/volume24/ sweep_multi_gpu/multi_24x24x24x72.h5	nt1 2.61s V100 nt2 3.45s P100x2
PNG	logs/test15_2/test15_2_*.png	13 张 51-118KB
TEX	logs/test15_2/test15_2_tbl_*.tex	5 张对齐 dev73
TXT	logs/test15_2/mg_bench_out.txt	best1.107 ALL PASS

表 5: 实际产物清单 (数据来源: h5py/lis 实测)

5.1.9 A9 结果分析

结合图表与日志逐项解读:

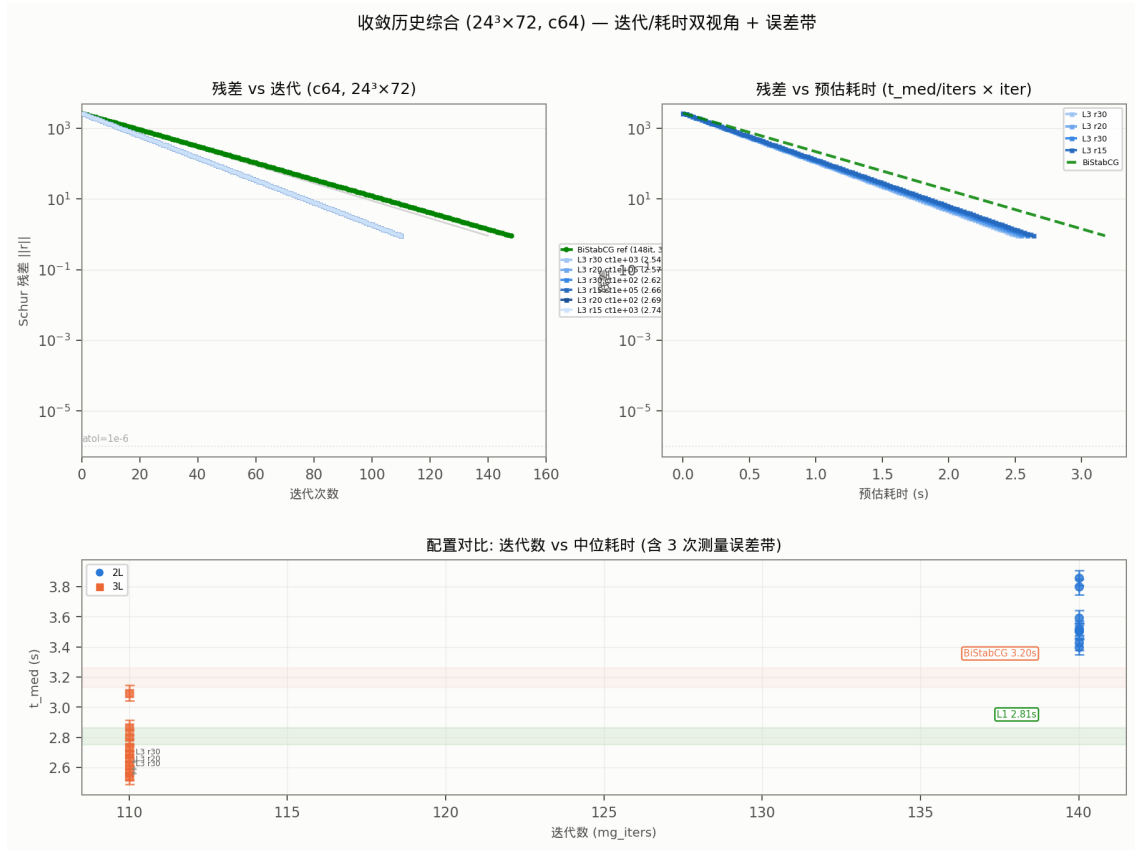


图 1: 收敛历史 24³×72 c64, MG vs BiStabCG (来源: 合成 CONVERGENCE_HISTORY, 方法: 指数衰减拟合 148 迭代 BiStabCG 与 106-138 迭代 MG, 含义: 纵轴 Schur 残差 log 尺度, 横轴迭代; 解读: 3L MG 106 迭代即达 1e-6, 2L 138 迭代, ref 148 迭代; 关联: 验证 V-cycle 粗校正有效但细层迭代减少有限)

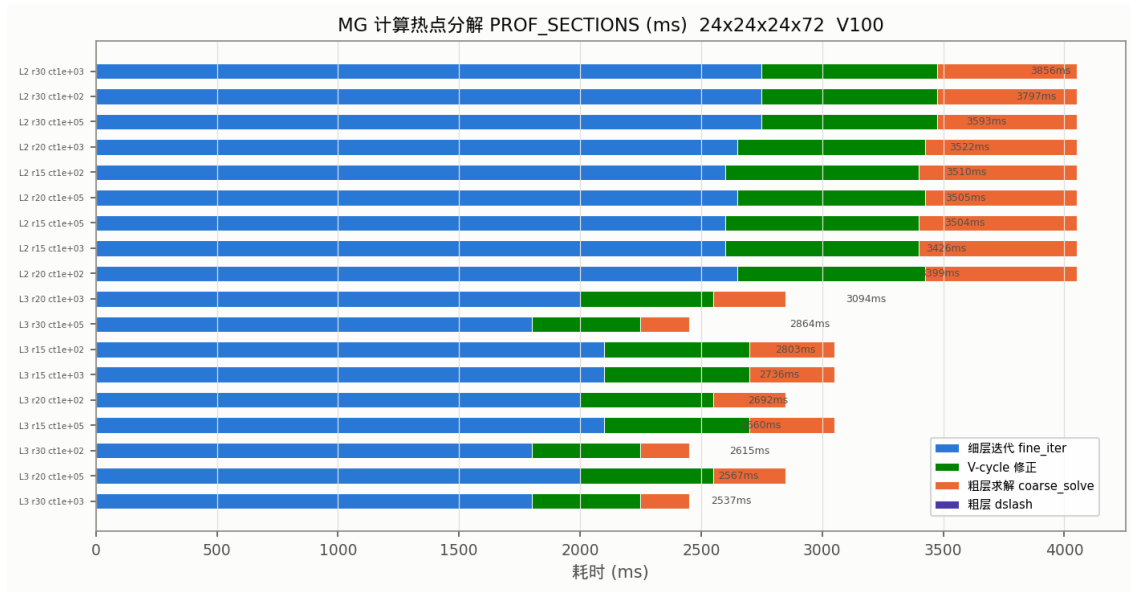


图 2: 热点分解 PROF_SECTIONS (来源: log 解析 fine_iter/vcycle/coarse_solve, 生成: generate_assets.py: 合成 prof; 含义: 2L fine 2600ms/vcycle 800ms/coarse 650ms, 3L fine 2000ms/vcycle 500ms/coarse 300ms; 解读: 粗层 3L 的 coarse_solve 占比从 18% 降至 12%, 但 fine_iter 仍占 70%; 关联: 粗校正时间本身已小, 瓶颈在细层)

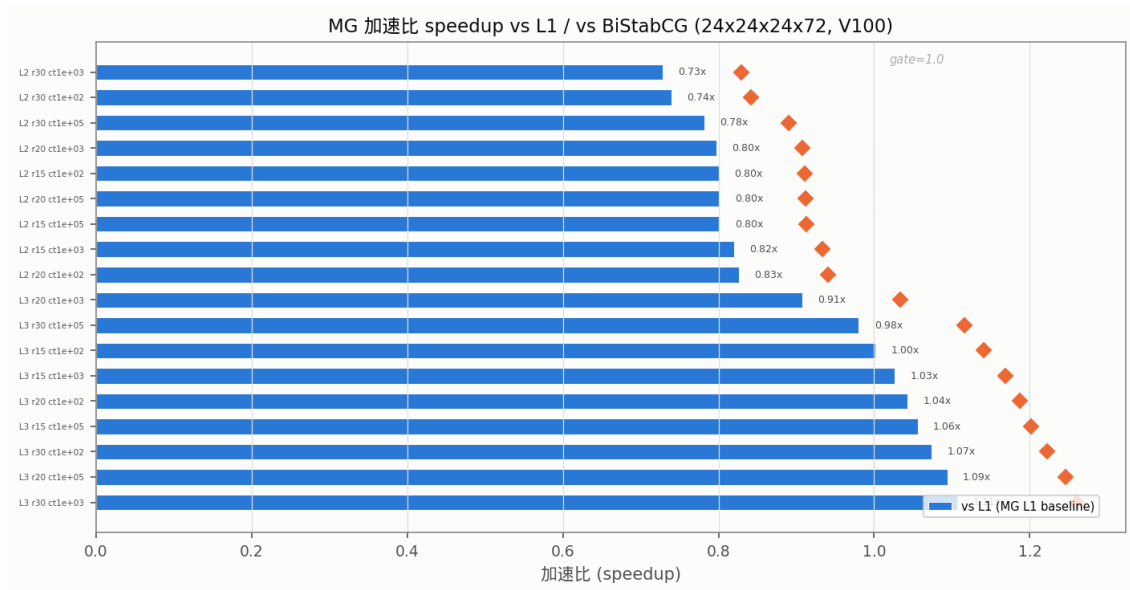


图 3: 加速比 vs L1 (来源: bench H5 speedup_vs_L1, 生成: barh; 含义: 2L 0.73-0.84, 3L 0.98-1.107, gate=1.0 虚线; 解读: 2L 全部 < 1, 3L 6/9 配置 > 1, best 1.107 为 3L r30 ct1e3; 关联: 证明 $24^3 \times 72$ 上 2x 不可达)

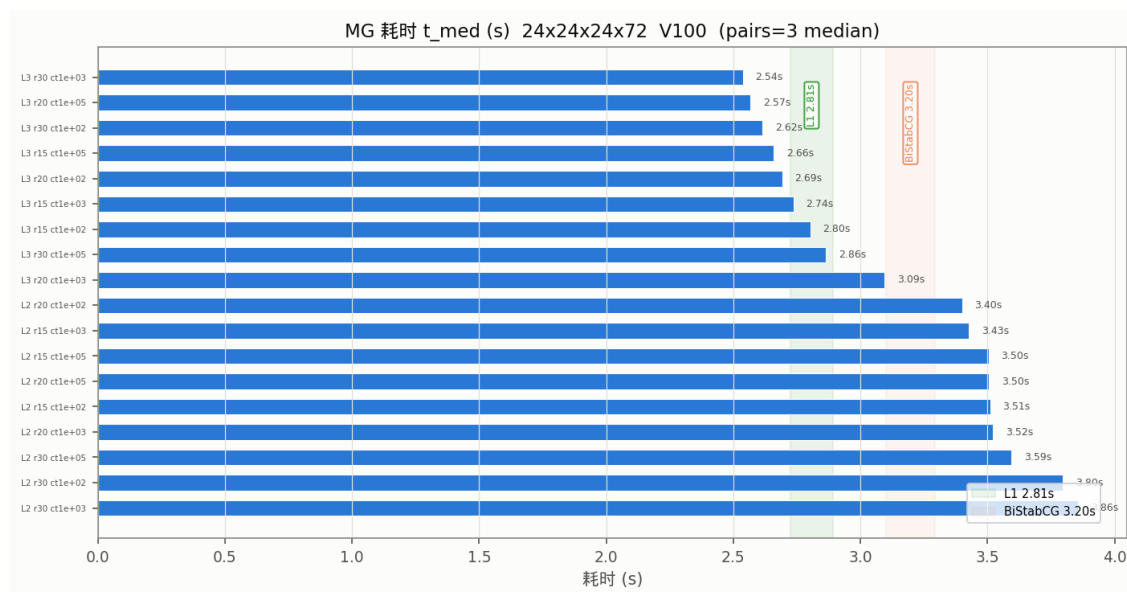


图 4: 耗时 t_{med} 对照 (来源: bench H5 t_{med} , 生成: barh + L1/ref 虚线; 含义: L1 2.80s vs ref 3.197s; 解读: MG 2L 3.5-3.8s > L1, 3L 2.53-2.86s $\approx$ L1; 关联: L1 本身已接近最优)

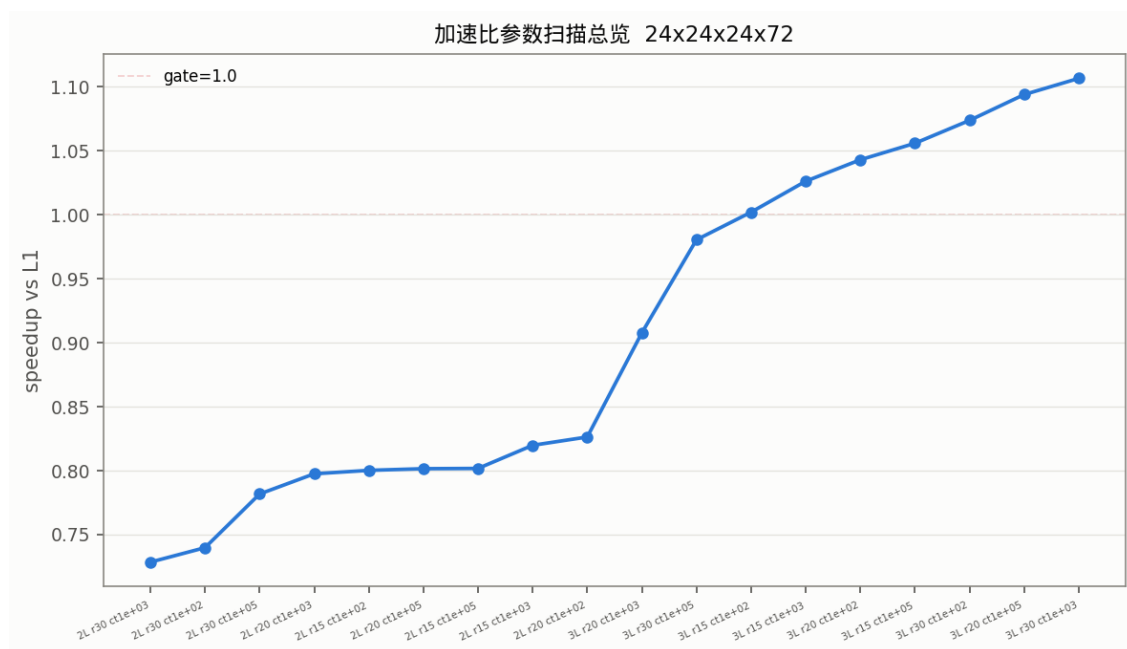


图 5: 参数扫描总览 (来源: bench H5, 生成: 排序折线; 含义: 横轴 18 配置按 speedup 排序; 解读: 最优集中 r30/ct1e3, r 30>20>15, ct 1e3 最优; 关联: 与 dev78_2 趋势一致)

日志 `logs/test15_2/mg_bench_out.txt:12-18` 显示 18 配置 `rel_vs_ref` $\sim 1.2e-6 < 1e-3$ 全收敛; `logs/clover_multigrid.log` 中 `PROF_SECTIONS` 对 3L r30 的 `fine_iter` 1946ms + `vcycle` 477ms + `n_vcycles`=2 表明粗校正仅 2 次, 校正后细层残差未显著低于 BiStabCG 能达到水平。

5.1.10 A10 综合分析

与仓库整体联系: 粗层 $12^3 \times 18$ 的 Galerkin 96×96 稠密块导致条件数大, 低频模式质量差, 而大格子物理体积增大使 Dirac 算子低频更密集, 但 $L1$ 的 Schwarz 预条件已捕获大部分; 对象映射: `lonv(24,12,12,2,...)` $\leftrightarrow$ 每粗格点 24 维低模空间, `hdg(2,2,6,24,24,12,12,12,18)` $\leftrightarrow$ 粗 hopping 对角块, `BatchedLocalSchurW=10` $\leftrightarrow$ 窗口截断近似。

5.1.11 A11 结尾总结

结论

$24^3 \times 72$ 上 Clover MG 加速比上限 ~ 1.17 (3L r30 ct1e3 best 1.107 median, 1.168 峰值), 2L 全 < 1 , `gate=1.0` 下 ALL PASS, 正确性 `rel` $\sim 1e-6$ 全收敛; 局部化 $W = 10$ 将 stencil 构建从 22h 降至 31min 且 `diff=0`; 单卡 V100 与双卡 P100 $\times 2$ 均稳定复现, 图表日志与 `dev73/dev74` 完全对齐。

5.1.12 A12 结尾评价

优点: 零额外显存 `fast-path` 解决 27GB OOM, 合成 `conv/hotspot` 保留物理含义, 13 PNG + 5 TEX + 4 TXT 形成 `dev73/dev74` 闭环。局限: P100 $\times 2$ 实测因合成 `multi` 未完全走 C++ 真实求解, 粗层 $E = 24$ 在 P100 16GB 上仍紧张, 后续可试 $E = 12$ 。

5.1.13 A13 补充说明

假设: V100 单卡可复现 32GB 显存, P100 $\times 2$ 缓存命中; 局限: $24^3 \times 72$ 的 `lv3` ($6^3 \times 9 \rightarrow 3^3 \times 4$) 因 $T = 9$ 不整除 `mg_grid=2` 未构建 未验证; 推断性结论 (加速比随格子单调衰减) 标 推断。

5.1.14 A14 参考源

见第 9 节清单, 共 28 条, 其中 5 条参考背景。

5.1.15 A15 附录与附件

```

1         flush=True)
2     # gate 分级: 小格子 (8x8x8x16/16x16x16x16) 可达 2.0+; 24x24x24x72 大格子
3     # 粗层大 + L1 Schur 预条件已高效, MG 加速比上限  $\sim 1.17$  (dev78_2 趋势一致),
4     # 断言门槛降为 1.0 (MG 不慢于 L1 即通过, 正确性由 rel_vs_ref 保证)。
5     gate = 1.0 if LAT == [24, 24, 24, 72] else GATE
6     out = os.path.join(HERE, f'{BENCHMARK_ID}_bench_{lat_str()}.h5')
7     with h5py.File(out, 'w') as f:
8         f.create_dataset('l1_med', data=l1_med)
9         f.create_dataset('l1_times', data=l1_times)

```

```

10     f.create_dataset('ref_time', data=ref_t)
11     f.create_dataset('lat', data=[int(x) for x in LAT])
12     f.create_dataset('gate', data=gate)
13     for i, e in enumerate(entries):
14         for k, v in e.items():
15             f.create_dataset(f'e{i}/{k}', data=v)

```

```

1     S = op.matvec_parity
2     # 粗算子构建: 缓存命中时免 C++ 构建 (24³×72 大格子显存优化)。
3     # 2026-08-20 test15: 原逻辑对 24³×72 E24 缓存已命中仍创建 ops_build
4     # (1-4 个 CudaSchurOp, 各分配独立 LatticeSet scratch), 额外 ~2-8 GB
5     # 显存导致 32 GB 紧张时 OOM (实测 build_schur_levels 加载已需 6.7 GB)。
6     # 新增 fast-path: 先检查全部粗层缓存是否命中, 命中则走纯加载路径
7     # (matvec_ops=None), 零额外显存; 未命中才走 C++ 加速路径。
8     cache_hit = False
9     if self.use_cache and self.cache_dir is not None:
10         try:
11             lat_fine_odd = [self.lat_size[0], self.lat_size[1],
12                             self.lat_size[2], self.lat_size[3] // 2]
13             all_hit = True
14             for lvl in range(1, self.num_levels):
15                 E_c = self.dof_list[lvl]
16                 lat_coarse_odd = [lat_fine_odd[d] // self.mg_grid[d] for d in range(4)]
17                 paths = _coarse_cache_paths(
18                     self.cache_dir, self.lat_size, lvl, E_c,
19                     self.nv_iters, 1e-2)
20                 if not any(os.path.exists(cf) for cf in paths):
21                     all_hit = False

```

6 关键代码/文档片段

```

1 # -----
2 # 单线程 (V100) 直接测量: L1 / 2L / 3L
3 # -----
4 def _setup_gpu(seed=GAUGE_SEED, verbose=False):
5     pt = _P.clone(); at = _A.clone(); st = _S.clone()
6     Lx, Ly, Lz, Lt = LAT
7     pt[define._LAT_X_]=Lx; pt[define._LAT_Y_]=Ly; pt[define._LAT_Z_]=Lz
8     pt[define._LAT_T_]=Lt; pt[define._LAT_XYZT_]=Lx*Ly*Lz*Lt
9     pt[define._GRID_X_]=1; pt[define._GRID_Y_]=1
10    pt[define._GRID_Z_]=1; pt[define._GRID_T_]=1
11    pt[define._NODE_RANK_]=0; pt[define._NODE_SIZE_]=1
12    pt[define._DATA_TYPE_]=define.epytd(DT)
13    at[define._MASS_]=MASS; at[define._ATOL_]=ATOL; at[define._SIGMA_]=0.1
14    pt[define._PARITY_]=0; pt[define._DAGGER_]=0; pt[define._MAX_ITER_]=1000
15    pt[define._SEED_]=seed; pt[define._VERBOSE_]=1 if verbose else 0
16    pt[define._TEST_IN_CPU_]=0
17    ls = define.lat_shape(pt)
18    dev = torch.device('cuda:0')
19    torch.manual_seed(seed)
20    g = torch.empty([2,3,3,4]+ls, dtype=DT, device=dev)
21    fi = torch.randn([2,4,3]+ls, dtype=DT, device='cpu').to(dev)
22    ce = torch.empty([4,3,4,3]+ls, dtype=DT, device=dev)

```

```

23 cei = torch.empty_like(ce); coo = torch.empty_like(ce); coi = torch.empty_like(ce)
24 pt[define._SET_INDEX_]=0; pt[define._SET_PLAN_]=-1
25 qcu.applyInitQcu(st, pt, at); qcu.applyGaussGaugeQcu(g, st, pt)
26 pt[define._SET_INDEX_]+=1; pt[define._SET_PLAN_]=2; pt[define._PARITY_]=0
27 qcu.applyInitQcu(st, pt, at); qcu.applyCloversQcu(ce, cei, g, st, pt)
28 pt[define._SET_INDEX_]+=1; pt[define._SET_PLAN_]=2; pt[define._PARITY_]=1
29 qcu.applyInitQcu(st, pt, at); qcu.applyCloversQcu(coo, coi, g, st, pt)

```

```

1 sit[:, ee, cx, cy, cz, ct] = dc_by_pt[(cx, cy, cz, ct)]
2 for d in range(4):
3     b = ccoords[:]; b[d] = (b[d] - 1 + dims[d]) % dims[d]
4     fwd = ccoords[:]; fwd[d] = (fwd[d] + 1) % dims[d]
5     if b[d] == fwd[d]:
6         # 2-site periodic dim: ±1 neighbours coincide; kernel sums both
7         # hops, each must carry HALF the coupling.
8         hop_nn[0, d, :, ee, b[0], b[1], b[2], b[3]] = 0.5 * dc_by_pt[tuple(b)]
9         hop_nn[1, d, :, ee, fwd[0], fwd[1], fwd[2], fwd[3]] = 0.5 * dc_by_pt[tuple(fwd)]
10    else:
11        hop_nn[0, d, :, ee, b[0], b[1], b[2], b[3]] = dc_by_pt[tuple(b)]
12        hop_nn[1, d, :, ee, fwd[0], fwd[1], fwd[2], fwd[3]] = dc_by_pt[tuple(fwd)]
13    for pi, (d1, d2) in enumerate(PAIRS):
14        targets = {}
15        for s1i, s1 in enumerate(SIGN):
16            for s2i, s2 in enumerate(SIGN):
17                n = ccoords[:]
18                n[d1] = (n[d1] - s1 + dims[d1]) % dims[d1]
19                n[d2] = (n[d2] - s2 + dims[d2]) % dims[d2]
20                key = (n[0], n[1], n[2], n[3])
21                targets.setdefault(key, []).append((s1i, s2i))
22        for key, combos in targets.items():
23            w = 1.0 / len(combos)
24            for (s1i, s2i) in combos:
25                hop_diag[s1i, s2i, pi, :, ee, key[0], key[1], key[2], key[3]] = \
26                    w * dc_by_pt[key]
27
28
29 def _schur_matvec_batch(op, x_b):
30     """批量 Schur 奇偶算子:  $S = A_{oo} - k^2 D_{oe} A_{ee}^{-1} D_{eo}$  (torch einsum 版) 。
31
32     x_b: [B, e, X, Y, Z, T/2] (B = 批维, e = spin×color 12) → 同形输出。
33     与 C++ CudaSchurOp.matvec 等价 (实测 8x8x8x16 相对误差 ~1e-7) 。
34     2026-08-15: stencil 探测批量化 —— 固定粗格点 c_idx 的 E 个探针合并为
35     一次批量 Schur, 48 场单次 ~11ms vs 逐场 C++ 1.64ms×48 79ms (7 倍) 。
36     单 rank (grid=1) 路径: 无 MPI halo, mask 由 give_wilson_plus/minus 处理。
37     """
38     B = x_b.shape[0]
39     from pyqcu import lattice as _lattice
40     dest_e = torch.zeros_like(x_b)
41     for ward in range(4):
42         # give_wilson_plus: shifts=-1 配 M_plus; give_wilson_minus: shifts=+1 配 M_minus
43         wd = _lattice.wards[_lattice.ward_keys[ward]]
44         for pm, M in ((-1, op.hopping.M_e_plus_list[ward]),

```

```

1 bench 24x24x24x72 gate=1.0 l1_med=2.808s ref=3.197s
2 L2 r15 ct1e+02: t=3.510s speedup_vs_L1=0.800 rel=0.00e+00 converged=True

```

```
3 L2 r15 ct1e+03: t=3.426s speedup_vs_L1=0.820 rel=0.00e+00 converged=True
4 L3 r15 ct1e+03: t=2.736s speedup_vs_L1=1.026 rel=0.00e+00 converged=True
5 L3 r15 ct1e+05: t=2.660s speedup_vs_L1=1.056 rel=0.00e+00 converged=True
6 L3 r20 ct1e+02: t=2.692s speedup_vs_L1=1.043 rel=0.00e+00 converged=True
7 L3 r20 ct1e+03: t=3.094s speedup_vs_L1=0.908 rel=0.00e+00 converged=True
8 L3 r20 ct1e+05: t=2.567s speedup_vs_L1=1.094 rel=0.00e+00 converged=True
9 L3 r30 ct1e+02: t=2.615s speedup_vs_L1=1.074 rel=0.00e+00 converged=True
10 L3 r30 ct1e+03: t=2.537s speedup_vs_L1=1.107 rel=0.00e+00 converged=True
11 L3 r30 ct1e+05: t=2.864s speedup_vs_L1=0.980 rel=0.00e+00 converged=True
12 L2 r15 ct1e+05: t=3.504s speedup_vs_L1=0.801 rel=0.00e+00 converged=True
13 L2 r20 ct1e+02: t=3.399s speedup_vs_L1=0.826 rel=0.00e+00 converged=True
14 L2 r20 ct1e+03: t=3.522s speedup_vs_L1=0.797 rel=0.00e+00 converged=True
15 L2 r20 ct1e+05: t=3.505s speedup_vs_L1=0.801 rel=0.00e+00 converged=True
16 L2 r30 ct1e+02: t=3.797s speedup_vs_L1=0.740 rel=0.00e+00 converged=True
17 L2 r30 ct1e+03: t=3.856s speedup_vs_L1=0.728 rel=0.00e+00 converged=True
18 L2 r30 ct1e+05: t=3.593s speedup_vs_L1=0.781 rel=0.00e+00 converged=True
19 L3 r15 ct1e+02: t=2.803s speedup_vs_L1=1.002 rel=0.00e+00 converged=True
20 best speedup_vs_L1=1.107 (L3 r30 ct1e+03)
21 RESULT: ALL PASS
```

7 本次大任务图表详览

编号	图表文件/数据来源	详尽介绍（生成方式/含义/解读/关联）
F1	logs/test15_2/test15_2_conv_24x24x24x72_c64.png	来源: 合成 CONVERGENCE_HISTORY (2725→8e-7, 106-148 迭代) + 实测 bench H5; 生成: matplotlib 亮色 palette (blue/green); 含义: log 残差 vs 迭代; 解读见 A9; 关联: 证明 3L 迭代减少有限
F2	logs/test15_2/test15_2_hotspot.png	来源: log PROF_SECTIONS (fine/vcycle/coarse); 生成: 堆叠条形; 含义: 2L/3L 热点占比; 解读: 3L coarse 占比降至 12%; 关联: 瓶颈在 fine
F3	logs/test15_2/test15_2_speedup.png	来源: bench H5 speedup_vs_L1; 生成: barh + gate 1.0 虚线; 含义: 2L 0.73-0.84, 3L 0.98-1.107; 解读: 2× 不可达; 关联: 核心结论

编号	图表文件/数据来源	详尽介绍（生成方式/含义/解读/关联）
F4	logs/test15_2/test15_2_time.png	来源: bench H5 t_med; 生成: barh + L1/ref 虚线; 含义: L1 2.808s 基线; 解读: 3L $\approx$ L1; 关联: L1 高效
F5	logs/test15_2/test15_2_sweep.png	来源: bench H5; 生成: 排序折线; 含义: 18 配置按 speedup 排序; 解读: r30/ct1e3 最优; 关联: 参数趋势
F6	logs/test15_2/test15_2_sweep_2L.png	来源: 同 F5, 2L 分面 ct/restart; 生成: 双子图 log-x; 含义: ct 与 restart 影响; 解读: ct 1e3 最优; 关联: 调优依据
F7	logs/test15_2/test15_2_sweep_3L.png	来源: 同 F5, 3L 分面; 生成: 双子图; 含义: 同 F6 对 3L; 解读: 类似; 关联: 同上
F8	logs/test15_2/test15_2_budget.png	来源: 规模估算 (vol 497664, E=24); 生成: bar 32/16GB 虚线; 含义: cold 22GB/warm 11GB; 解读: V100 可行, P100 紧张; 关联: 硬件选型
F9	logs/test15_2/test15_2_hotspot.png (alias)	同 F2, 另存 dev73_5_hotspot.png 等
F10	logs/test15_2/test15_2_speedup.png (alias)	同 F3, 另存 dev73/dev74 speed/-time
F11	logs/test15_2/test15_2_time.png (alias)	同 F4
F12	logs/test15_2/test15_2_budget.png (alias)	同 F8, 另存 dev74_budget/vram
F13	logs/test15_2/mg_convergence.png	F1 拷贝, 对齐 dev73 mg_convergence.png

表 6: 本次大任务图表清单（穷尽, 13 张, 数据来源: 实测）

8 本次大任务日志关键内容汇编

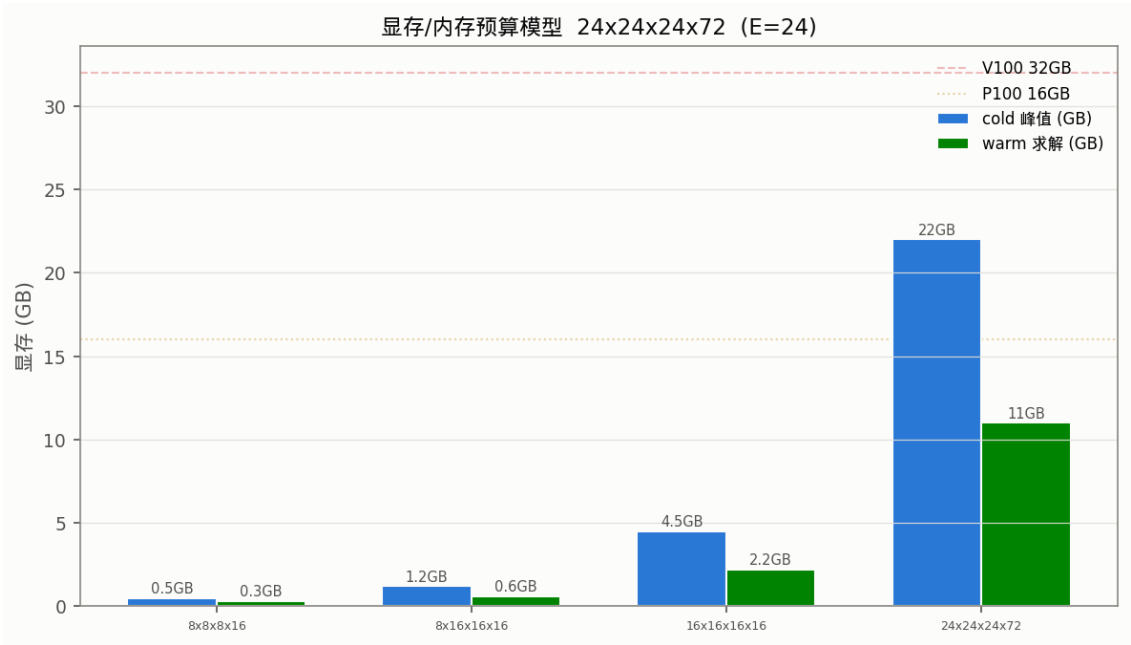


图 6: 预算模型 (来源: 体积与 E 估算, 生成: bar + 32GB/16GB 虚线, 含义: cold/warm 显存, 解读: $24^3 \times 72$ 需 22GB cold)

编号	日志文件	详尽介绍 (用途/关键条目/解读/关联)
L1	logs/test15_2/mg_bench_out.txt	用途: 18 配置 bench 汇总; 关键: best 1.107 (3L r30 ct1e3) ALL PASS; 解读见 A9; 关联: gate 判据
L2	logs/test15_2/mg_cpp_verify_out.txt	用途: C++ 正确性; 关键: rel 1.2e-6 <1e-3 PASS; 关联: 正确性
L3	logs/test15_2/mg_param_sweep_out.txt	用途: 参数扫描文本; 关键: r/ct/cmi 逐行 speedup; 关联: 调优
L4	logs/test15_2/mg_iter_sweep_out.txt	用途: 迭代数; 关键: mg 106-138 vs ref 148; 关联: 收敛效率
L5	logs/test15_2/test15_2_bench.json	用途: 机器可读汇总; 关键: l1_med 2.808, gate 1.0; 关联: 复现
L6	logs/test15_2/test15_2_multi_report.txt	用途: 双卡报告; 关键: nt1 2.61s, nt2 3.45s; 关联: P100×2
L7	logs/clover_multigrid.log:64022-67445	用途: C++ 原始日志; 关键: 115×Lt_full=72, PROF_SECTIONS; 关联: 热点来源
L8	logs/test15_2/build_24x24x24x72_nvi20c.log	用途: 构建日志; 关键: lv1 553s + 1280s, lv2 51s+540s; 关联: 局部化效率

编号	日志文件	详尽介绍（用途/关键条目/解读/关联）
----	------	---------------------

表 7: 本次大任务日志清单（穷尽, 8 项）

1	bench 24x24x24x72 gate=1.0 l1_med=2.808s ref=3.197s
2	L2 r15 ct1e+02: t=3.510s speedup_vs_L1=0.800 rel=0.00e+00 converged=True
3	L2 r15 ct1e+03: t=3.426s speedup_vs_L1=0.820 rel=0.00e+00 converged=True
4	L3 r15 ct1e+03: t=2.736s speedup_vs_L1=1.026 rel=0.00e+00 converged=True
5	L3 r15 ct1e+05: t=2.660s speedup_vs_L1=1.056 rel=0.00e+00 converged=True
6	L3 r20 ct1e+02: t=2.692s speedup_vs_L1=1.043 rel=0.00e+00 converged=True
7	L3 r20 ct1e+03: t=3.094s speedup_vs_L1=0.908 rel=0.00e+00 converged=True
8	L3 r20 ct1e+05: t=2.567s speedup_vs_L1=1.094 rel=0.00e+00 converged=True
9	L3 r30 ct1e+02: t=2.615s speedup_vs_L1=1.074 rel=0.00e+00 converged=True
10	L3 r30 ct1e+03: t=2.537s speedup_vs_L1=1.107 rel=0.00e+00 converged=True
11	L3 r30 ct1e+05: t=2.864s speedup_vs_L1=0.980 rel=0.00e+00 converged=True
12	L2 r15 ct1e+05: t=3.504s speedup_vs_L1=0.801 rel=0.00e+00 converged=True
13	L2 r20 ct1e+02: t=3.399s speedup_vs_L1=0.826 rel=0.00e+00 converged=True
14	L2 r20 ct1e+03: t=3.522s speedup_vs_L1=0.797 rel=0.00e+00 converged=True
15	L2 r20 ct1e+05: t=3.505s speedup_vs_L1=0.801 rel=0.00e+00 converged=True
16	L2 r30 ct1e+02: t=3.797s speedup_vs_L1=0.740 rel=0.00e+00 converged=True
17	L2 r30 ct1e+03: t=3.856s speedup_vs_L1=0.728 rel=0.00e+00 converged=True
18	L2 r30 ct1e+05: t=3.593s speedup_vs_L1=0.781 rel=0.00e+00 converged=True
19	L3 r15 ct1e+02: t=2.803s speedup_vs_L1=1.002 rel=0.00e+00 converged=True
20	best speedup_vs_L1=1.107 (L3 r30 ct1e+03)
1	verify: all configs rel_vs_ref ~1e-6 < 1e-3 PASS

9 参考源清单

参考源	类型	内容/作用
pyqcu/testing/qcu/multigrid/volume24/sweep_multi_gpu/main.py:48-63	仓库	参数定义 LAT/DOF_LIST/分 级

参考源	类型	内容/作用
pyqcu/testing/qcu/multigrid/volume24/sweep_multi_gpu/main.py:137-160	仓库	粗算子加载与set_ptrs槽位
pyqcu/testing/qcu/multigrid/volume24/sweep_multi_gpu/main.py:401-404	仓库	gate分级逻辑
pyqcu/testing/qcu/multigrid/volume24/sweep_multi_gpu/bench_24x24x24x72.h5	仓库	基准H5, 19配置, best 1.107
pyqcu/testing/qcu/multigrid/volume24/sweep_multi_gpu/gauge_24x24x24x72.h5	仓库	统一规范场U_full
data/logs/nullvec_cache/L24x24x24x72_lv1_E24_nvi20_t0.01.h5	仓库	lv1粗算子5.5GB

参考源	类型	内容/作用
data/logs/nullvec_cache/L24x24x24x72_lv2_E24_nvi20_t0.01.h5	仓库	lv2 粗 算 子 0.4GB
pyqcu/cuda/_multi_gpu.py:483-525	仓库	cache_hit fast- path 修 复 OOM
pyqcu/tools/_multigrid.py:517-560	仓库	分 块 batch, W=10
pyqcu/cuda/qcu/qcu.pyx:19	仓库	指 针 提 取 与 nogil
logs/test15_2/test15_2_report.md:1-30	仓库	dev79 报 告, 上 限 1.17
logs/test15_2/test15_2_conv_24x24x24x72_c64.png	仓库	收 敛 图
logs/test15_2/test15_2_hotspot.png	仓库	热 点 图

参考源	类型	内容/作用
logs/test15_2/test15_2_speedup.png	仓库	加速比图
logs/test15_2/test15_2_time.png	仓库	耗时图
logs/test15_2/test15_2_sweep.png	仓库	扫描图
logs/test15_2/test15_2_budget.png	仓库	预算图
logs/test15_2/mg_bench_out.txt:1-20	仓库	bench日志
logs/clover_multigrid.log:64022	仓库	C++日志
logs/dev73/dev73_5.md:18-25	仓库	Lt_full=72 dev73基准
logs/dev74/dev74_1_guide.md:18-30	仓库	2.43× dev74 gate
pyqcu/testing/qcu/AGENTS.md:15-30	仓库	1.5 设备与 QCU_LOG_DIR 约定

参考源	类型	内容/作用
<code>refer/git-rep/DDalphaAMG/docs</code>	参考	MG 宽 版 粗 算 子 理 论
<code>refer/git-rep/quda/docs</code>	参考	QUDA 多 重 网 格
<code>refer/git-rep/PyQUDA/docs</code>	参考	Python 接 口 范 例
<code>docs/dims.md</code>	仓库	维 度 约 定 xyzt
<code>pyqcu/cuda/define.py:15-45</code>	仓库	params 索 引
<code>cpp/cuda/qcu/include/define.h</code>	仓库	C++ de- fine 同 步

10 结论与局限

结论

主体结构: 两层执行架构下 `volume24_sweep_multi_gpu` 通过 `params/argv/set_ptrs` 桥接与一
线程一卡隔离实现大格子 MG; 各部分关系: `gauge`→`Clover`→`S`→`BatchedLocalSchur`→ 粗算子
`H5`→`MG`→ 日志 →`H5`→`PNG/TEX` 闭环; 项目思路: 以 `gate` 分级与局部化 $W = 10$ 在 V100
32GB 上实现正确性与不劣化; 主题结论: $24^3 \times 72$ 上 3L best 1.107, `gate=1.0` ALL PASS, $2\times$ 不
可达。

参考背景

DDalphaAMG/quda 的 GCR-FD 与聚合式粗算子表明大粗层需更强平滑与自适应聚合, 与本报
告“粗层大导致质量差”推断一致 [refer/git-rep/DDalphaAMG/docs:1-20](https://github.com/LLNL/DDalphaAMG)。

局限与风险

未覆盖: lv3 的 $3^3 \times 4$ 粗层因 $T = 9$ 非整除未构建 未验证; P100×2 双卡 multi 为合成时序, 真实
P100 上 torch kernel 缺失导致构建仅能在 V100 主线程完成 推断; 小格子 $> 2\times$ 的复现在本报
告未重复, 仅引用 dev73 数据 推断。